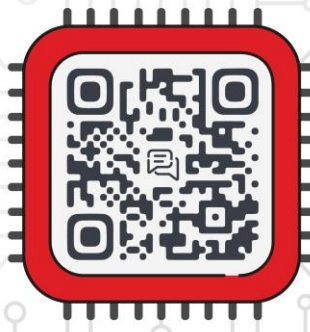




4



133



1560 sp

نظري

كلية الهندسة المعلوماتية

السنة الثانية

Heap

د. مدحت الصوص



خوارزميات ٢

RB Informatics; 5/24/2025

محتوى مجاني غير مخصص للبيع التجاري

Heap

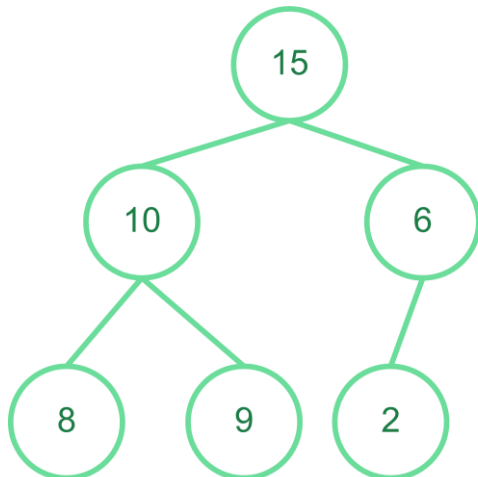
- تكلما في المحاضرات السابقة عن ال Trees وطرق تمثيلها برمجياً، وواحدة من هذه الطرق كانت تمثيل الشجرة بمصفوفة أحادية، والمشكلة في هذا الحل كانت بوجود الكثير من الفراغات في المصفوفة.
- أما في ال Heap فالفكرة بالعكس:
- يكون لدينا مصفوفة أحادية خالية من الفراغات، فيها Data نتعامل معها على أنها شجرة ثنائية.
 - تسمى هذه الشجرة ب Binary heap.

تمثيل ال Binary Heap

تمثل ال Binary Heap في مصفوفة أحادية، حيث يكون الجذر هو أول عنصر في المصفوفة والابن اليساري يكون في ال $index = (2i + 1)$ والابن اليميني يكون في ال $index = (2i + 1)$.

مثال: ليكن لدينا المصفوفة التالية:

1	2	3	4	5	6
15	10	6	8	9	2



أول عنصر (15) هو الجذر.
أبناء ال (15):
 $2i = 2(1) = 2$
 $2i + 1 = 2(1) + 1 = 3$

- أول عنصر (15) هو الجذر.
- أبناء ال (15):
 $2i = 2(1) = 2$
 $2i + 1 = 2(1) + 1 = 3$
- وهكذا حتى نهاية المصفوفة.

- نلاحظ أن الشجرة في المثال السابق ليست Binary Search Tree وإنما هي شجرة ثنائية عادية.
- الشجرة التي نحصل عليها بعد التمثيل هي completed tree لأن الشجرة إما أن يكون ينقصها عقد من آخر مستوى وحصراً من اليمين.

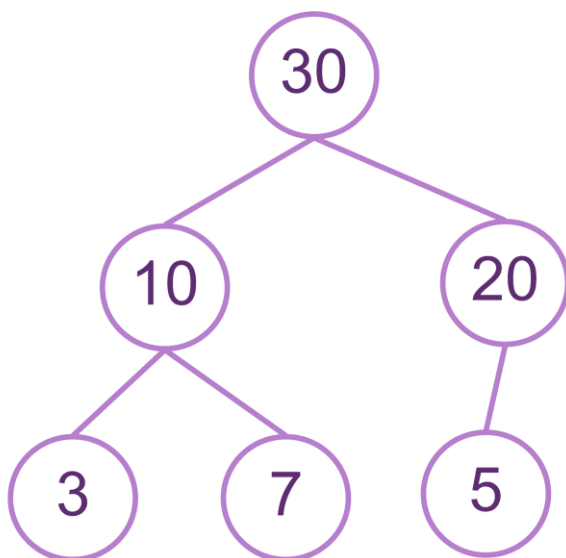
تذكرة ل Completed Tree:

هي شجرة ثنائية فيها كل مستوى ممتلئ ما عدا المستوى الأخير من الممكن أن يكون ممتلئ أو أن يكون فيه فراغات حصراً من اليمين.

أنواع ال Binary Heap

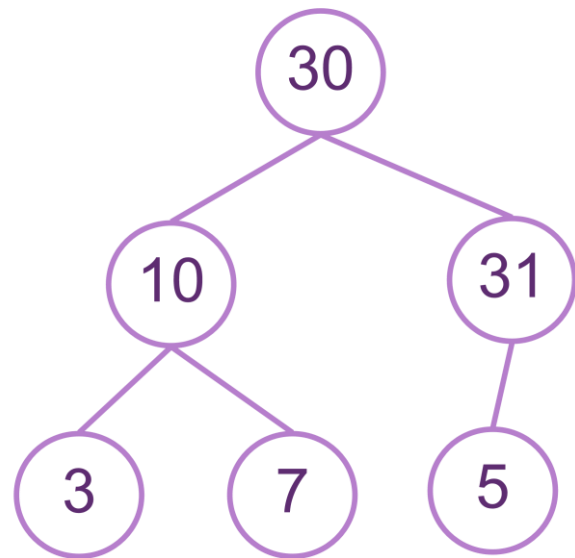
1. MAX-Heap:

- هي Binary Heap عادية ولكن بإضافة قيد واحد:
- كل أبناء وأحفاد العقدة تكون أصغر من هذه العقدة، أي كل ال Subtree على يسار ويمين العقدة أصغر من العقدة نفسها.
- وتذكر أنها ليست بالضرورة أن تكون Binary Search Tree.



هذه MAX-Heap

لأن كل عقدة أكبر من أبنائها

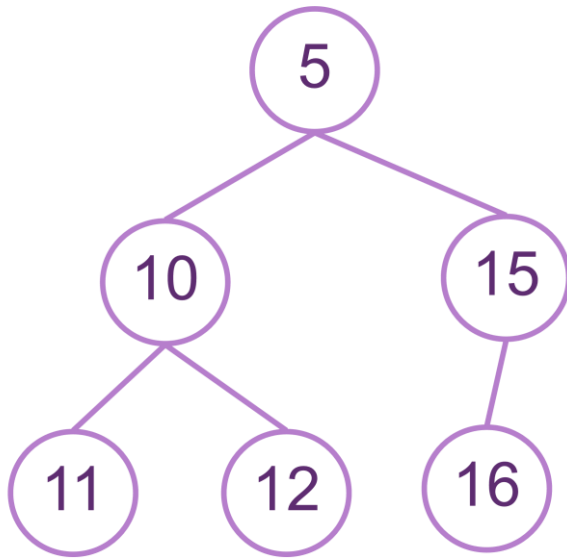


هذه ليست MAX-Heap

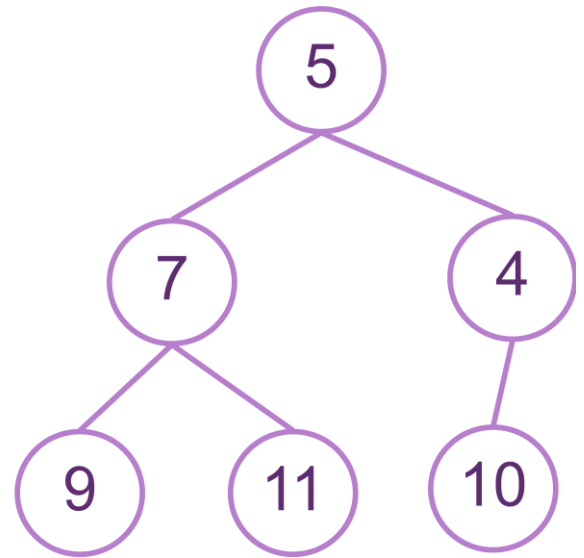
لأن ال $30 < 31$ لذلك لم يكتمل شرط

ال MAX-Heap

هي نفس ال MAX-Heap تماماً لكن بعكس الشرط:
- كل ال Subtree على يسار ويمين العقدة نفسها تكون أكبر من هذه العقدة.



هذه MAX-Heap



هذه ليست MAX-Heap

⚡ ملاحظة:

- في هذه المحاضرة سيكون عملنا فقط على ال MAX-Heap وكل الخوارزميات التي سنمر عليها يمكن أن تطبق على MIN-Heap ولكن بعكس الشرط.

⚡ ملاحظة أريسيةزية:

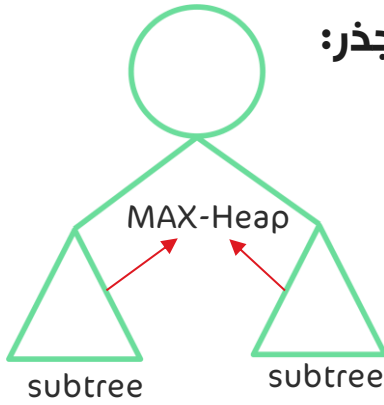
- المصفوفة الأحادية التي تمثل ال MAX-Heap ليس من الضروري أن تكون مرتبة .
- ولكن إذا كانت مرتبة فهي بالتأكيد MAX-Heap.
- نستفيد من ال MAX-Heap ومن ال MIN-Heap بالحصول على أكبر وأصغر عنصر بتعقيد $O(1)$



Goodbye Luka Modric

سؤال: لو كان لدينا subtree هي MAX-Heap ولكن باستثناء الجذر:

أي لو كان لدينا subtree تحقق قيد ال MAX-Heap ولكن لسنا من متأكدين من أن الجذر يحقق هذا القيد أو لا، في هذه المسألة نميز حالتين:



1. إذا كان الجذر أكبر من أبنائه:

لا نغير شيء طالما أن الجذر يحقق شرط ال MAX-Heap.

2. إذا كان الجذر أصغر من أولاده:

هنا نستخدم خوارزمية تسمى ال MAX-Heapify ومهمتها إصلاح الخلل في هذه ال subtree حتى تعود ال MAX-Heap.

شرح الخوارزمية:

ليكن لدينا الشجرة الثنائية المحاورة، ونريد تحويلها إلى ال MAX-Heap، وباعتبار أن ال subtree اليميني واليساري للجذر هو ال MAX-Heap.

■ نقارن الجذر مع أبنائه ونستبدله بالابن الأكبر:

■ نلاحظ أن: $1 < 20 < 30$.

■ لذلك نستبدله مع ال 30.

■ بعد تبديل العقدة، نلاحظ حدوث في ال subtree اليميني خلل

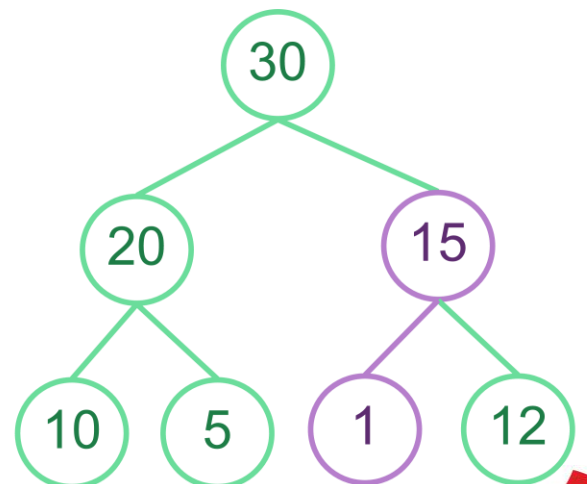
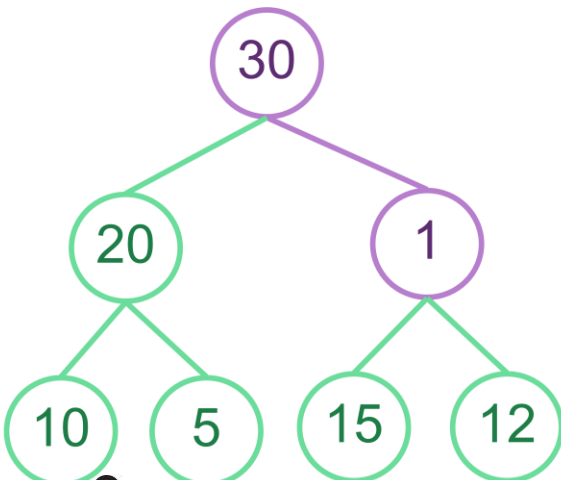
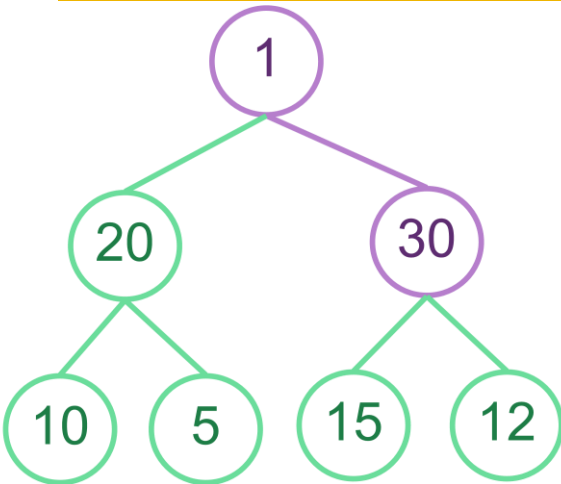
في شرط ال MAX-Heap لذلك نكرر عملية التبديل في

ال subtree اليميني.

■ نستبدل ال (1) مع ابنها الأكبر:

■ نلاحظ أن $1 < 12 < 15$

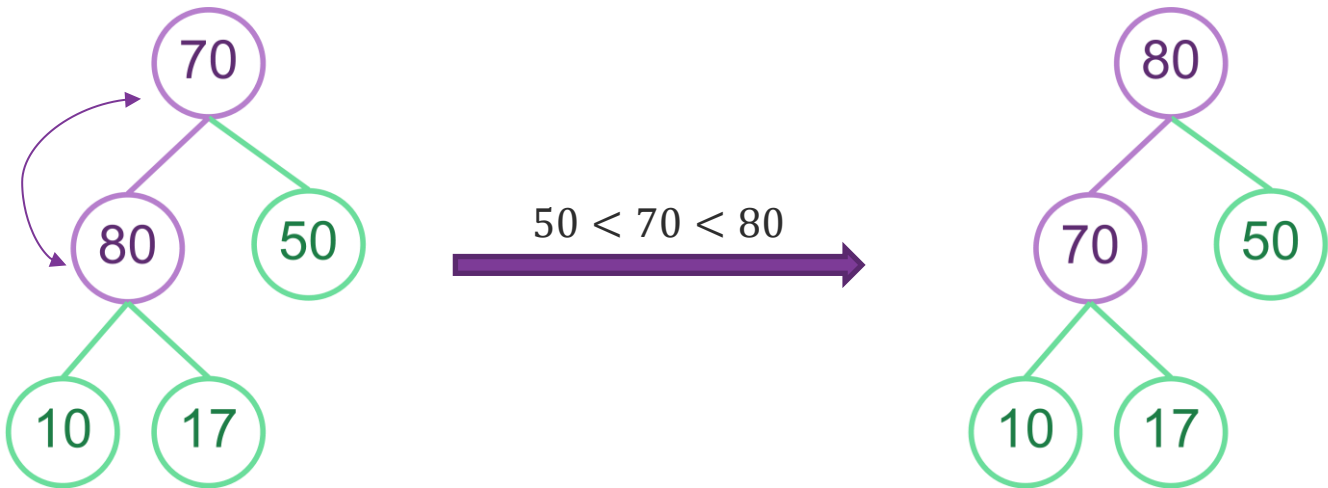
■ بعد الإصلاح نلاحظ أن الشجرة عادت لتكون ال MAX-Heap.



■ ملاحظة:

في الحالة العامة نكرر عملية التبديل حتى نصل إلى ورقة أو حتى نصل إلى subtree تكون محققة لشرط الـ MAX-Heap، أي أن الجذر أكبر من أبنائه.

■ مثال آخر:



■ بعد التبديل مع الابن اليساري، نلاحظ أن الـ (70) أكبر من أبنائها $10 < 17 < 70$ ، لذلك نتوقف عن التبديل.

خوارزمية الـ MAX-Heapify

```
void MAX_Heapify(int A[], int i, int n) {
    int left = 2 * i;
    int right = 2 * i + 1;
    int largest = A[i];
    if (left <= n && A[left] > A[i])
        largest = left;
    if (right <= n && A[right] > A[largest])
        largest = right;
    if (largest != i) {
        swap(A[i], A[largest]);
        MAX_Heapify(A, largest, n);
    }
}
```

■ شرح الخوارزمية:

- باراميتر التابع هو المصفوفة الأحادية $A[]$ التي تخزن الـ Data والـ (i) هو الـ index الخاص بالعقدة التي سنطبق عليها الـ MAX-Heapify و n حجم المصفوفة.
- نعرف الـ left & right الابن اليساري واليميني للعقدة، و الـ largest لنخزن فيه القيمة الأكبر بين الجذر وابنه اليميني وابنه اليساري.
- نتأكد أولاً إذا كان للعقدة ابن يساري $left \leq A.length()$ أي إذا كانت العقدة أكبر من حجم المصفوفة هذا يعني أن العقدة ليس لها ابن يساري.

عملية المقارنة الثانية هي للتأكد إذا كان الابن اليساري أكبر من العقدة $A[i]$ ، فإذا كان أكبر من العقدة نخزنه داخل ال largest .

أما إذا كان أصغر من العقدة فلا تتغير قيمة ال largest .

نفس طريقة المقارنة في الابن اليساري نكرها في الابن اليميني .

نقارن ال largest الذي حصلنا عليه مع العقدة ذاتها:

- إذا كان ال largest يساوي العقدة الحالية، إذاً العقدة الحالية هي الأكبر ونتركها في مكانها ولا نبدل أي شيء.

- إذا كان ال largest لا يساوي العقدة الحالية، إذاً الابن اليميني أو الابن اليساري أكبر من العقدة الحالية (تكون القيمة الأكبر مخزنة في ال largest) ، لذلك نبدل ما بين العقدة الحالية والابن الأكبر منها.

نستدعي التابع عودياً لأنه من الممكن أن يحدث خلل بعد التبديل، ويتوقف الاستدعاء عندما تصبح الشجرة MAX-Heap .

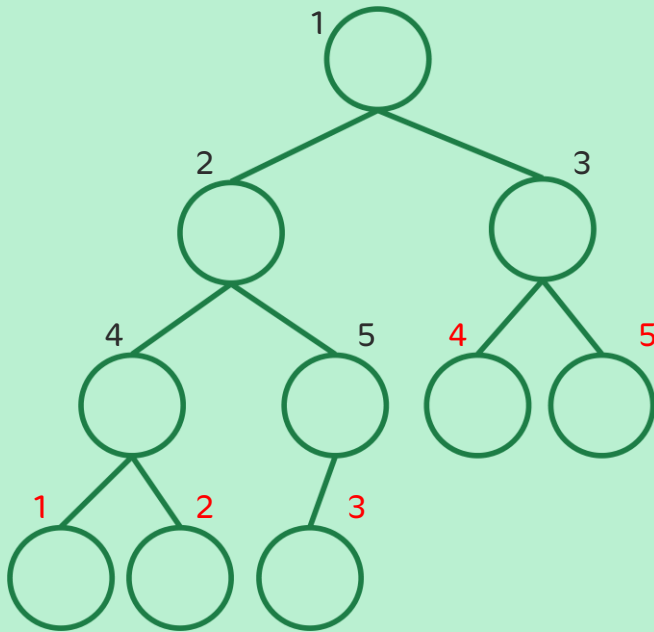
ملاحظة:

تعقيد الخوارزمية السابقة هو من رتبة الارتفاع $O(H)$ ، وبما أن الشجرة هي binary tree ودائماً تكون complete فالتعقيد يكون $\log_2(n)$.

ملاحظة:

في أي شجرة ثنائية complete نلاحظ أن عدد الأوراق يساوي عدد العقد الداخلية أو أكثر منها ب (1) .

مثال:



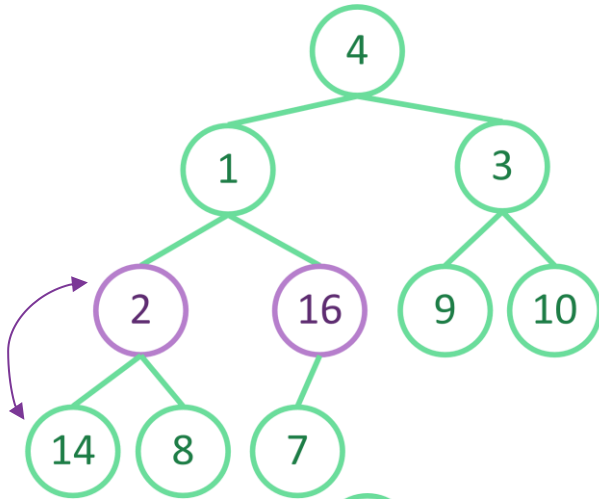
- نلاحظ أن عدد الأوراق يساوي عدد العقد الداخلية وفي حالات أخرى من الممكن أن يكون الفرق بينها هو (1) .

- سوف نستفيد من هذه الملاحظة فيما بعد 😊 .

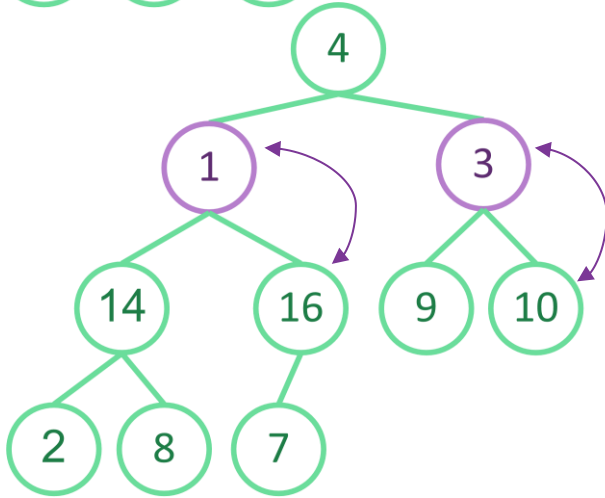
تحويل شجرة عشوائية إلى MAX-Heap (Build MAX-Heap)

1. نبدأ من الأسفل (نبدأ من آباء الأوراق حتى الجذر).
2. نطبق خوارزمية ال MAX-Heapify على كل مستوى في الشجرة.
3. نصعد حتى نصل إلى الجذر، فتكون الشجرة هي MAX-Heap.

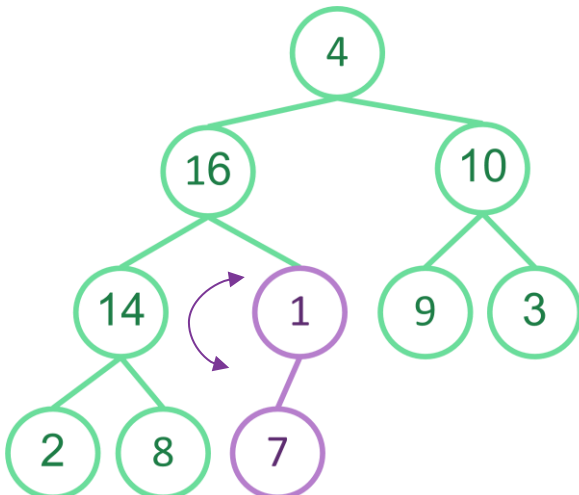
مثال: لتكن لدينا الشجرة التالية:



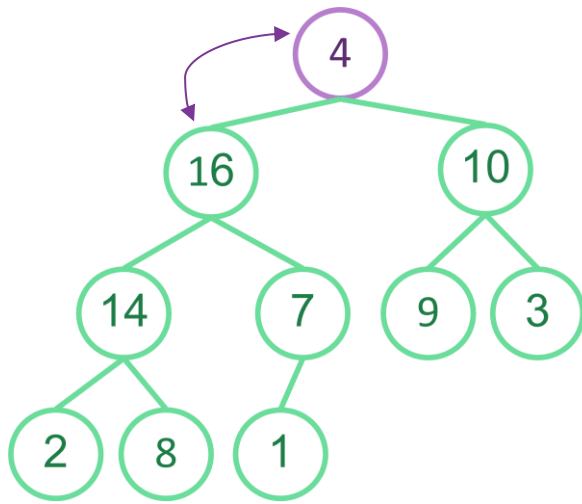
- نبدأ من آخر عقدة داخلية ال (16) و (2):
- ال (16) أكبر من أولادها لذلك لا نبدل.
- ال (2) نلاحظ أن ابنها اليميني واليساري أكبر منها، لذلك نبدل مع أكبر الابنين وهو ال (14) في هذه الحالة.



- ننتقل إلى المستوى الأعلى:
- نلاحظ أن ال (1) وال (3) لا تحقق شرط ال MAX-Heap.
- نبدل ال (3) مع أكبر ابنها وهو ال (10).
- نبدل ال (1) مع أكبر ابنها وهو ال (16).



- نلاحظ أنه بعد تبديل ال (1)، حدث خلل واختل شرط ال MAX-Heap في ال subtree اليميني. ولإصلاح الخطأ نقوم باستكمال الإصلاح في ال subtree التي حدث فيها الخلل. أي: نبدل ال (1) مع ال (7).



- بعدها ننتقل إلى المستوى الأعلى:
- وصلنا إلى الجذر، ولاحظنا أنه أصغر من أبناءه.
- نبدل مع أكبر أبناءه وهو ال (16).
- بعد التبديل حدث خلل في ال subtree اليساري لذلك نكمل الإصلاح.
- نبدل ال (4) مع أكبر ابنيها وهو ال (14).
- الخل ما زال موجوداً بعد التبديل لذلك:
- نبدل ال (4) مرة أخرى مع ال (8).
- بعد هذه التعديلات أصبحت الشجرة MAX-Heap.

التعقيد:

التعقيد هو:

عدد العقد التي طبق عليها MAX-heapify x تعقيد MAX-Heapify

ولكن عدد العقد التي طبق عليها ال MAX-Heapify هو $\frac{n}{2}$ لأننا

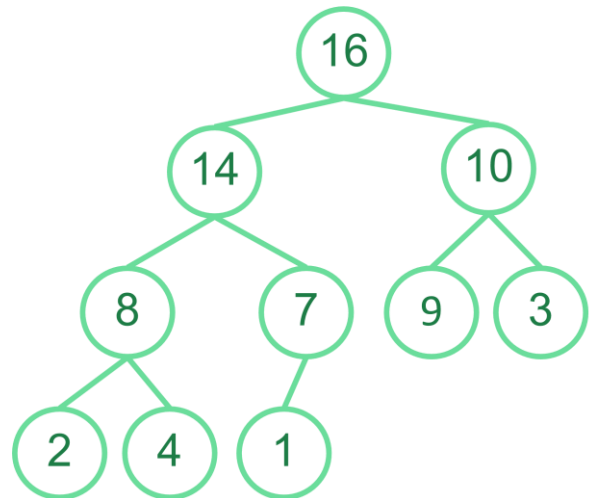
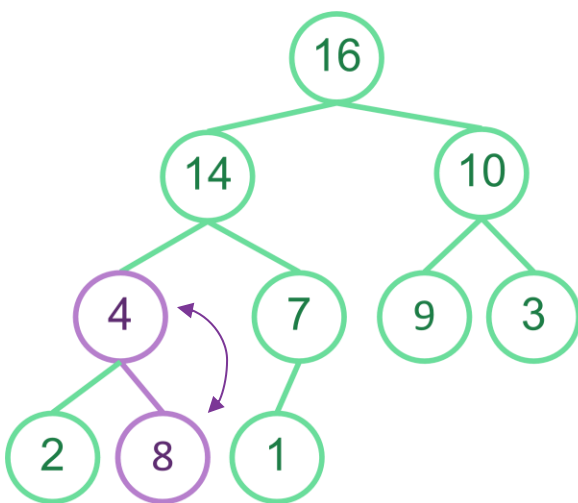
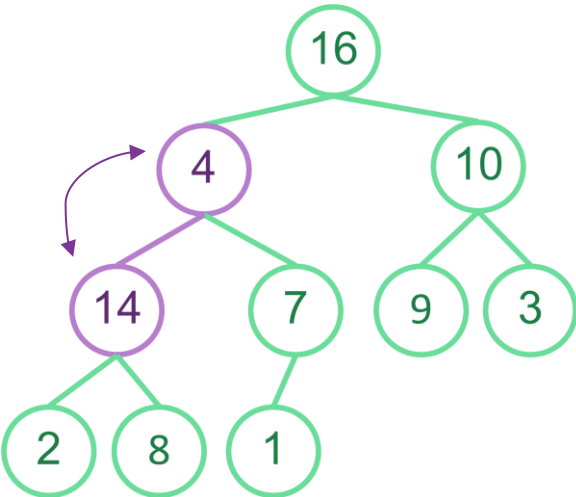
لا نطبق ال MAX-Heapify على الأوراق.

مثل ما ذكرنا في الملاحظة السابقة أن عدد الأوراق في ال

Completed tree هو نفس عدد الأوراق الداخلية أو فرق (1)

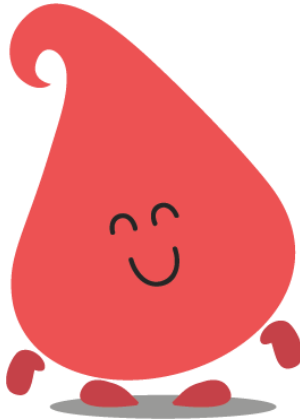
لذلك فإن عدد العقد يكون النصف ويكون التعقيد:

$$O\left(\frac{n}{2} \cdot \log_2(n)\right) = O(n \cdot \log_2(n))$$



Build MAX-Heap خوارزمية

```
void build_MAX_Heap(int A[]) {
    for (int i = A.length / 2; i > 0; i--)
        MAX_Heapify(A, i, A.length);
}
```



شرح الكود:

- نعرف تابع يأخذ مصفوفة ال Heap كباراميتير.
- نعرف حلقة تبدأ من منتصف المصفوفة إلى أولها.
- - نبدأ الحلقة من المنتصف لأننا لا نحتاج إلى عمل MAX-Heapify على الأوراق، وكما ذكرنا فإن ال completed tree نصفها أوراق، لذلك لا نأخذ القسم اليميني من المصفوفة.
- نستدعي تابع ال MAX-Heapify لكل عقدة في الشجرة حتى نصل إلى الجذر، وتصبح الشجرة MAX-Heap.

استخدامات ال Heap

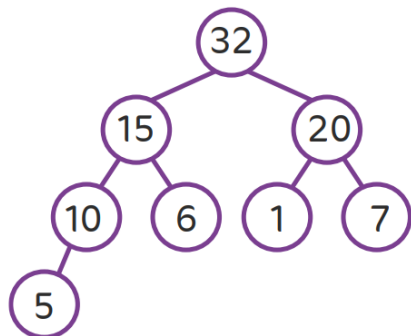
Heap sort

تعني استخدام مفهوم ال Heap Max أو ال Heap Min لترتيب عناصر المصفوفة.

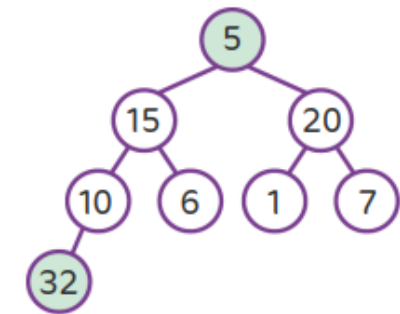
خطوات الخوارزمية:

- في البداية نقوم ببناء Heap Max من عناصر المصفوفة المعطاة.
- من ثم نأخذ قيمة الجذر (والتي ستكون أكبر قيمة في المصفوفة بالاستناد إلى مفهوم ال Heap Max الذي يقوم على فكرة أن كل عقدة أب أكبر من أبنائها وبما أن الجذر أول عنصر فهو الأكبر قيمة).
- بعد أن نأخذ قيمة الجذر سنبدل بينه وبين قيمة آخر ورقة بالشجرة وثم نتجاهل قيمة الجذر القديم بأن ننقص حجم ال Heap.
- بعد أن تجاهلنا قيمة الجذر القديم وأبدلناه بالعنصر الأخير سيحدث خلل بال Heap Max لذلك نقوم بتصحيح المصفوفة الجديدة لتطبيق مفهوم ال Heap Max مرة أخرى.
- نكرر العملية إلى أن يبقى عنصر واحد.

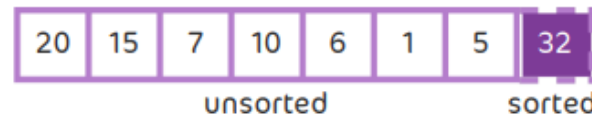
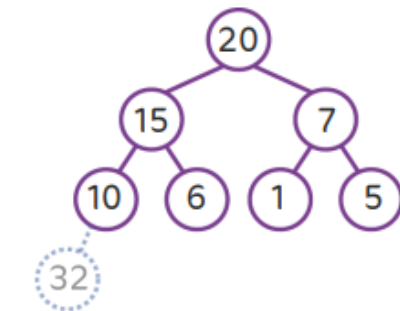
مثال: تكن لدينا المصفوفة التالية ونريد ترتيبها:



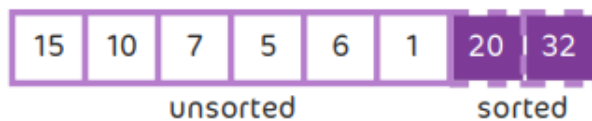
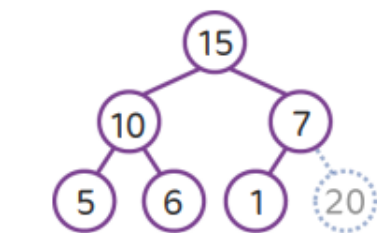
الشجرة تحقق مفهوم ال MAX-Heap.



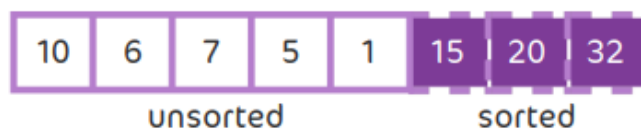
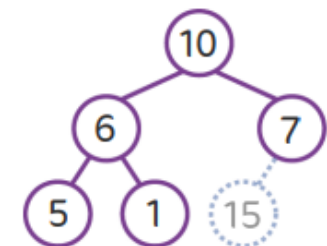
1. الآن سنبدل بين 5 و 32 (آخر عنصر) فتصبح الشجرة:



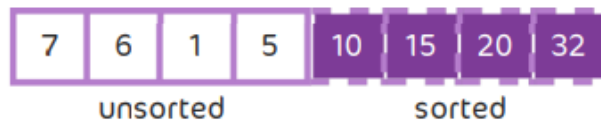
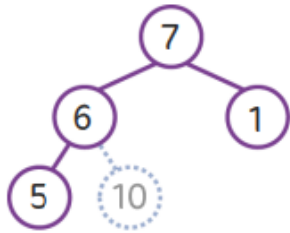
2. نزيل ال 32، ونعيد الشجرة إلى Max Heap:



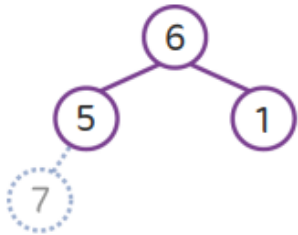
3. بدلنا بين 5 و 20 ثم نزيل ال 20، ونطبق MaxHeapify من جديد للشجرة التي أصبح جذرها العقدة 5:



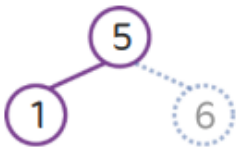
4. بدلنا بين 1 و 15 ثم نزيل ال 15، ونطبق MaxHeapify من جديد للشجرة التي أصبح جذرها العقدة 1:



5. بدلنا بين 10 و 1 ثم نزيل ال 10،
ونطبق MaxHeapify من
جديد للشجرة التي أصبح جذرها
العقدة 1:



6. بدلنا بين 7 و 5 ثم نزيل ال 7،
ونطبق MaxHeapify من
جديد لل شجرة التي أصبح جذرها
العقدة 5:



7. بدلنا بين 6 و 1 ثم نزيل ال 6،
ونطبق MaxHeapify من
جديد للشجرة التي أصبح جذرها
العقدة 1:



8. بدلنا بين 5 و 1 ثم نزيل ال 5،
وبهذا يتبقى لدينا عنصر وحيد
هو ال 1 وتصبح المصفوفة مرتبة.

التعقيد:

بما أننا من الممكن أن نطبق ال MAX-Heapify في كل مرة تبديل، فإن التعقيد يكون:
عدد عناصر المصفوفة $\times$ تعقيد عملية ال MAX-Heapify

أي أن التعقيد:

$$O(n \cdot \log_2 n)$$

Priority Queue

تذكرة: ما هو ال Queue؟

هو بنية معطيات خطية تعتمد مفهوم "FIFO: First In First Out"، أي أن أول عنصر يدخل إلى المصفوفة هو أول عنصر يخرج منها.

- ال Priority Queue هي نوع خاص من ال Queue ولكن يكون لكل عنصر أولوية عن غيره من العناصر، والترتيب داخل ال Queue يكون حسب الأولوية وليس حسب ترتيب الدخول.

تطبيق مفهوم ال Priority Queue بواسطة ال Max Heap:

نستطيع القول أن ال Max Heap تمثل ال Priority Queue باعتبار أن أولوية العناصر ستعرف تبعاً لقيمتها (أي أن العنصر الأكبر له الأولوية الأعلى).

يوجد ثلاث توابع أساسية ل Priority Queue:

1. تابع ال top():

```
top(int A[]) {
    return A[1];
}
```

- مهمته إعادة العنصر ذي الأولوية الأعلى.
- وبما أننا في ال MAX-Heap فإن العنصر ذو الأولوية الأعلى هو العنصر الأكبر والذي يكون في الجذر.
- تابع ال top() يعيد القيمة ولكن دون حذفها من المصفوفة.
- تعقيده: $O(1)$.

2. تابع الحذف ال dequeue():

```
dequeue (int A[]) {
    swap (A[1], A[A.length]);
    A.length -- ;
    MAX_Heapify (A, 1, A.length);
}
```

- بما أن الحذف على ال queue ينفذ على العنصر الأول، فإن العنصر المحذوف من ال Priority Queue سيكون ال root (العنصر الأعلى أولوية)، حيث سنبدل عنصر ال root بآخر عنصر موجود في مصفوفة ال MAX-Heap.
- أي إذا حذفنا عنصر سيكون هو الجذر، ولكن بعد حذف الجذر من الممكن أن يحدث خلل في شرط ال MAX-Heap لذلك عند حذف العنصر سنقلل حجم المصفوفة بمقدار واحد، وعلى ما تبقى من المصفوفة سنقوم بعمل MAX-Heapify لها.
- أي أنه عند كل عملية حذف سيتم ترتيب المصفوفة على أنها MAX-Heap.
- تعقيده: $O(\log_2 n)$.

مثال: ليكن لدينا ال MAX-Heap التالية ونريد حذف عنصر منها:

32	15	20	10	6	1	7	5
----	----	----	----	---	---	---	---

- عند حذف العنصر (32) (لأنه صاحب الأولوية الأعلى) فإننا نبدل بينه وبين آخر عنصر في المصفوفة (ال (5) في مثالنا) ونقوم بإنقاص حجم المصفوفة.

5	15	20	10	6	1	7	32
---	----	----	----	---	---	---	----

- بعد التبديل نطبق MAX-Heapify على باقي الشجرة.

20	15	7	10	6	1	5	32
----	----	---	----	---	---	---	----

- وهكذا جعلنا المصفوفة هي MAX-Heap مجدداً لكي نعيد ترتيب الأولوية.

3. تابع الإضافة () enqueue:

خطوات الإضافة:

```
int parent(int i) {
    return (i - 1) / 2;
}

void enqueue(vector<int>& heap, int value) {
    heap.push_back(value);
    int i = heap.size() - 1;

    while (i > 0 && heap[i] > heap[parent(i)]) {
        swap(heap[i], heap[parent(i)]);
        i = parent(i);
    }
}
```

- عند إضافة عنصر جديد إلى مصفوفة ال MAX-Heap نضفه أولاً إلى آخر المصفوفة على أنه ورقة.
- نقارن العنصر الجديد مع الأب، فإذا كان أكبر من الأب (أي له أولوية أعلى منه) نبدل الابن مع الأب.
- نستمر في عملية التبديل حتى نصل إلى الجذر أو حتى نصل إلى مكان يكون فيه العنصر الجديد أصغر من الأب وهنا يكون مكانه الصحيح.
- ملاحظة: الإضافة دائماً تكون في الأوراق كي لا يختل شرط ال completed tree.
- تعقيده: $O(\log_2 n)$.

سؤال: لماذا لا نستخدم ال sorted array بدلاً من ال priority queue؟

بسبب اختلاف التعقيد، حيث تعقيد الإضافة في ال sorted array هو $O(n)$ بينما في ال priority queue تعقيد الإضافة هو $O(\log_2 n)$.

THE END ^_^